

KERNEL MASTERS

Linux System Programming

Inter Process Communication

Authored and Compiled By: Boddu Kishore Kumar

Email: kishore@kernelmasters.org

Reach us online: www.kernelmasters.org

Contact: 9949062828

Important Notice

This courseware is both the product of the author and of freely available open source materials. Wherever external material has been shown, its source and ownership have been clearly attributed. We acknowledge all copyrights and trademarks of the respective owners.

The contents of this courseware cannot be copied or reproduced in any form whatsoever without the explicit written consent of the author.

Only the programs - source code and binaries (where applicable) - that form part of this courseware, and that are present on the participant CD, are released under the GNU GPL v2 license and can therefore be used subject to terms of the afore-mentioned license. If you do use any of them, in any manner, you will also be required to clearly attribute their original source (author of this courseware and/or other copyright/trademark holders).

The duration, contents, content matter, programs, etc. contained in this courseware and companion participant CD are subject to change at any point in time without prior notice to individual participants.

Care has been taken in the preparation of this material, but there is no warranty, expressed or implied of any kind and we can assume no responsibility for any errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

2012-2022 Kishore Kumar Boddu
Kernel Masters, Hyderabad - INDIA.

Introduction to IPC.

Processes executing concurrently in the operating system may be either **independent processes** or **cooperating processes**.

Independent process	Cooperating process
A process is independent if it cannot affect or be affected by the other processes executing in the system.	A process is cooperating if it can affect or be affected by the other processes executing in the system.
Any process that does not share data with any other process is independent.	Any process that shares data with other processes is a cooperating process.

There are several reasons for providing an environment that allows process cooperation:

- **Information sharing.** Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information.
- **Computation speedup.** If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing elements (such as CPUs or I/O channels).
- **Modularity.** We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.
- **Convenience.** Even an individual user may work on many tasks at the same time. For instance, a user may be editing, printing, and compiling in parallel.

Cooperating processes require an inter process communication (IPC) mechanism that will allow them to exchange data and information. There are two fundamental models of inter process communication: **(1) shared memory** and **(2) message passing**. In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region. In the message passing model, communication takes place by means of messages exchanged between the cooperating processes. The two communications models are contrasted in Figure 1.1.

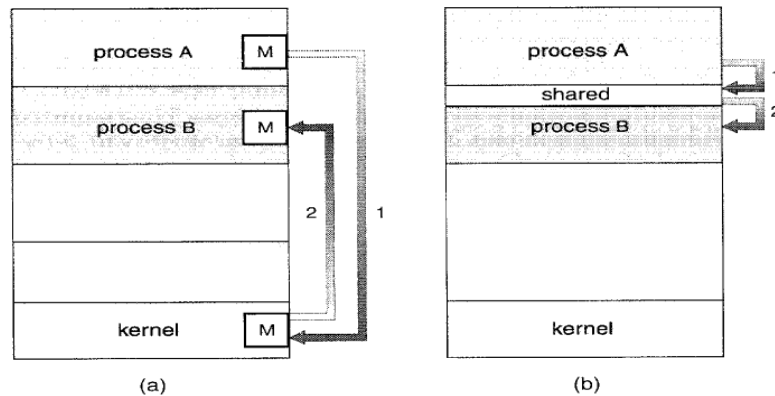


Fig 1.1: Communication models. (a) Message Passing (b) Shared Memory

Message Queues	Shared Memory
Message Queues slower than Shared memory.	Shared memory is faster than message passing.
Message-passing systems are typically implemented using system calls and thus require the more time-consuming task of kernel intervention.	Shared-memory systems, system calls are required only to establish shared-memory regions. Once shared memory is established, all accesses are treated as routine memory accesses, and no assistance from the kernel is required

1. Pipes:

- **Connection oriented:** Writing or reading is possible only after connection is established between two processes. Processes should open the connection before reading or writing.
- Simplest form of IPC.
- Pipe is called as un-named pipe.
- Can be used in shell.

Advantages:

- No message boundary.

Drawbacks:

- Pipe is useful only among processes related as parent/child.
- Only for related process.
- Cannot store data.

2. Named Pipes (FIFO):

- FIFO is called as named pipe.
- Can store data.
- If there are multiple receivers there is not way to direct the data to particular user.

Advantages:

- Two unrelated processes can use FIFO unlike plain pipe.
- Can be used for non-related processes.

Drawbacks:

- Less secure than pipe, since any process with valid access permission can access them.
- No message boundaries. Data is treated as stream of byte

3. Message Queues:

- **Connection less:** A process can send data without caring for the connection between two process. So sender can send message, even if no receiver is present is connection less mechanism.
- Can direct and store data.

Advantages:

- Message queue is a dedicated IPC without any file management.
- Message queues and signals can be great for hard real-time applications, but they are not as flexible.
- Effective for small amount of data.

Drawbacks:

- Very expensive for large data, as transferred through kernel buffer and each data transfer involves 2 data copy operations.
- Message queue is a slowest IPC mechanisms.
- Message boundary is defined.
- Message is deleted after reading.

4. Shared Memory:

- Fastest communication among all the IPC events.
- No synchronization like pipe or FIFO.
- Race condition is possible so locking is required.
- When next message will come with different id, then the previous message will be deleted.

Advantages:

- This can be a **very fast method** of information transfer because RAM can be used.
- Data are not removed after reading.

Drawbacks:

- **Synchronization is a major problem - if the first process keeps writing data, how can it ensure that the second one reads it?**
- Data can either read or written only. Append is not possible.

5. Sockets:

Sockets are more like ports that you can send data to. A process will be "listening" at a port and will accept data sent to it.